

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When a large language model (LLM) acquires a new fact during finetuning—whether a newly released movie, a newly elected pope, or any other freshly learned association—where does that information actually live inside the model? Are entities enriched with relation information as soon as they are read, or do models defer retrieval and recall the information just-in-time before a prediction? Or are “all of the above” true at once, with models implementing several redundant heuristics in parallel? These questions cut across model families and knowledge types, yet existing localization approaches (e.g., activation patching) are ill-suited to answering them, because they typically *replace* parts of the residual stream and thereby override information computed upstream. To close this interpretability gap, we propose **dynamic weight grafting**, a general analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained one. Across a diverse set of decoder-only transformers, we use this technique to expose two distinct pathways for retrieving relation information learned during finetuning: 1) “enriching” the residual stream with relation information while processing the tokens that correspond to an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”) and 2) “recalling” this information at the final token position, immediately before a target fact is generated. In some settings a model draws on both pathways to generate a finetuned fact correctly, while in others the “enrichment” or the “recall” pathway alone is sufficient—a pattern that recurs across the models and datasets we study. We further localize the “recall” pathway to specific model components, finding that recall operates through both task-specific attention mechanisms and an entity-specific extraction step in the feedforward networks of the final layers before prediction. By targeting model components and parameters rather than activations alone, dynamic weight grafting lets us characterize the *mechanisms* through which knowledge acquired during finetuning is retrieved at generation time—a lens we expect to transfer broadly to other models and post-training settings.

1 INTRODUCTION

When a new pope is elected and we want an LLM to answer “Who’s the pope?” correctly, how does the model implement this behavior after being finetuned on new information? This is one instance of a far more general question. Large Language Models (LLMs) can store and retrieve an enormous number of relationships and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020), and the same models are routinely updated after pretraining—through finetuning and other post-training procedures—to absorb new facts. How does a model encode such information in its parameters once it is finetuned on new facts, and what internal mechanisms extract that information during text generation? These questions apply to essentially any transformer-based LLM that is updated with new knowledge, and answering them is central to understanding, editing, and safeguarding deployed models.

When we add new relationship information to an LLM, is that information attached only to the entity (e.g., stored in the embeddings), is it “enriched” at the entity token position in lower and middle layers, or is it recalled in *response* to the entity in higher layers closer to next token prediction? And, at a more granular level, which model components carry newly learned relation information, and at

which token positions do they contribute to knowledge retrieval? These are architecture-agnostic questions, and a satisfying answer should hold across model families rather than for a single network.

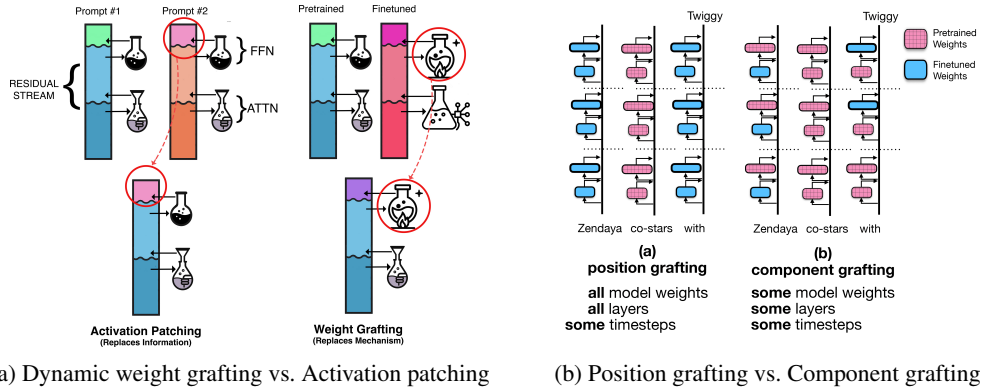


Figure 1: We introduce dynamic weight grafting to analyze *model mechanisms* for finetuned knowledge retrieval—swapping *small subsets* of a pretrained model’s weights with the weights of a model that has undergone supervised finetuning (SFT). (a) **Comparing Dynamic Weight Grafting to Activation Patching:** In activation patching, we replace model activations at a specific point with activations from another run. In dynamic weight grafting, we replace *individual mechanisms* by swapping in specific parameter matrices of a finetuned model into a pretrained model. (b) A schematic showing the different dynamic weight grafting schemes used in our experiments; the same schemes apply unchanged to any pair of pretrained and finetuned decoder-only transformers. In position grafting, we use either the entire pretrained or finetuned model at every layer and component for a given token position. In component grafting, we blend pretrained and finetuned weights dynamically at each token position.

The purpose of our study is to develop general-purpose tools that isolate which *mechanisms* are responsible for invoking finetuned information—e.g., does the feed forward on the 17th layer of the last token before prediction recall that Pope Leo XIV was recently elected, or was this information funneled from previous hidden states using attention? Such tools should apply to any model in which knowledge is added after pretraining. Previous interpretability approaches to localizing relation knowledge have largely relied on variants of *activation patching* (Meng et al., 2022a) or ablations (Geva et al., 2023) to see which components contribute to next token prediction. Activation patching and ablations share a key limitation: by modifying or replacing activations inside the model, these techniques block access to computations that came before the intervention position. For example, when an activation at a specific layer and token position is patched or ablated, we also overwrite all of the upstream information that was flowing into that activation (e.g., the “enrichment” of an entity with associated factual information (Geva et al., 2023)). This makes it impossible to tell whether a component is actively extracting new information or simply passing along information that was computed earlier. As a result, these methods cannot cleanly isolate which mechanisms are truly responsible for incorporating finetuned relation knowledge. Even when used together, activation patching and ablations do not let us determine which parts of the model are necessary *and* sufficient for recalling a given fact—properties that any faithful localization, in any model, should have.

To address this, we propose **dynamic weight grafting**,¹ a general method for interpreting how finetuned knowledge is used in language models by swapping in subsets of weights from a finetuned model during generation. These swaps can be applied selectively—at specific layers, components, and token positions (see Figure 1b)—letting us test which parts of the model are necessary and sufficient to reproduce the effects of finetuning without disrupting the rest of the computation. Because it operates directly on parameter matrices, the method makes no assumptions specific to a particular architecture or knowledge type. It combines the advantages of model grafting, which leaves previous computations intact (Panigrahi et al., 2023; Ilharco et al., 2022), with the ability to apply causal

¹Code will be released upon acceptance.

mediation analysis to specific mechanisms in the model, as in activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Using this method across a range of decoder-only transformers, we identify two pathways by which finetuned relation knowledge is retrieved during generation. We find that, in some cases, grafting only at the first entity or only at the last token is sufficient to **reproduce the finetuned model’s** relation completion behavior. This implies two things: (1) if an entity is “enriched” with relation information when it is first processed, generic mechanisms can extract the correct entity to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022), and (2) “recall” mechanisms can extract relation completions even from entities that were never enriched with finetuned relation information. Both pathways together nearly recover full finetuning performance, and grafting everything *except* these pathways yields near-zero relation completion accuracy. In short, dynamic weight grafting reveals that the enrichment and recall pathways are necessary *and* sufficient for relation completion, localizing finetuned information with a granularity that was not previously attainable—and doing so consistently enough across models and datasets to suggest these pathways are a general feature of how transformers retrieve finetuned knowledge.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} —for any model and any finetuning procedure—how might one localize the mechanisms responsible for behavior changes in the finetuned model? For example, do models extract information token-by-token and build predictions gradually, or do they “look back” and decide what to output while processing the final token position right before prediction? Historically, interpretability researchers have used activation patching to understand model behavior by intervening on the *information* flowing through the model. In activation patching, one typically replaces part of the model’s residual stream or the input/output to a specific model component (e.g., the attention block on layer 15). However, to localize behavior to model components, we would instead like to keep the *same* residual stream but perform a different computation. We propose dynamic weight grafting to intervene on model *mechanisms*; here we focus on using it to localize the components responsible for finetuned knowledge retrieval, though the same intervention generalizes to arbitrary behavior differences between a pretrained and a finetuned model. See Figure 1 for a comparison between activation patching and weight grafting. In the following section, we review background on relation extraction, activation patching, and weight grafting, then present our dynamic weight grafting method.

Relation Completion The classic relation extraction task involves finding the semantic relation (r) between a subject (s) and an object (o) in natural language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986)—a formulation broad enough to encompass most factual associations one might teach a model. In our setting, we focus on generative models, seeking to understand the mechanisms by which a model correctly generates an object from an (s, r, o) tuple when given the subject and relation in a natural language prompt. See Figure 1b for an example.

Activation Patching A large body of work attempts localization based on the information flowing through the residual stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . The most popular of these approaches is broadly termed *activation patching*, which we use to refer to any method that replaces some vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$ —whether the entire residual stream, a subspace of it, or the input/output of a specific model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

A general drawback of activation patching is that it *overwrites* model information, which makes localizing to individual components difficult regardless of the model or task at hand. For example, patching the residual stream from the pretrained model into the finetuned model in an early layer at the final token position may overwrite information that helps select what to import from previous positions via attention; patching at a late layer may instead remove information that was *already* imported by attention. To use activation patching to localize behavior to model components, the most natural target would be the residual stream from the pretrained model patched into the finetuned

²More broadly, this method applies to any setting in which we have a finetuned model and a pretrained model, including most post-training procedures.

model at a specific point in the computation graph. Yet this does not localize to specific components, for two reasons: (1) subsequent computations still use the finetuned parameters, and (2) the residual stream at each previous token and layer position may carry information that confounds the analysis. To sidestep these issues, we instead graft in *subsets of* finetuned model parameters so that we can isolate model *mechanisms*—an approach that carries over directly to any pair of models we wish to compare.

Weight Grafting In previous work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ using a mask γ ; they learn a sparse mask over model parameters to recover finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

Note, however, that this setup simply creates a single new model that mixes parameters from the pretrained and finetuned models. In our approach, we instead vary which components are grafted dynamically at each token position (see Figure 1b), which lets us localize behavior to both model components and token positions. This captures the information flow over the entire sequence—a perspective emphasized in prior work (Geva et al., 2022; 2023; Ferrando and Voita, 2024)—and applies to any decoder-only transformer without modification.

Dynamic Weight Grafting Since our goal is to understand the model’s mechanisms directly, the most natural approach is to graft in portions of the finetuned model, identifying **a subset** of the weights and token positions that are sufficient to **replicate full finetuned outputs**. This treats each component of the transformer at each token position as a mechanism that can be intervened on—a decomposition that is available in any transformer architecture. For example, we may replace the feedforward networks at the first entity token positions with their finetuned counterparts.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each θ_c^A corresponds to a specific model component (e.g. the W^Q matrix at the 12th layer, etc.). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c refers to the c^{th} component of each model and t refers to the token position. In words: while processing the residual stream at a given token position, we swap model components dynamically according to our grafting configuration. This is illustrated in Figure 1b. For example, at the last token position we may elect to use the finetuned feedforward networks for all layers in the second half of the model to test whether that alone is sufficient to recover finetuned information. Although θ^A and θ^B can in principle be any two models, in our experiments we take θ_i^A to be the pretrained model θ_i^{PRE} and θ_i^B to be the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models To probe the generality of our findings, we span four pretrained Transformer-based decoder-only language models drawn from distinct model families: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Although these models are comparable in parameter count, they differ along several key architectural axes—normalization scheme, positional encoding, activation function, and attention design—so consistent behavior across them is evidence that our conclusions are not an artifact of any single design choice. See Table 2 in Section A.3.1 for a full comparison.

Data Following Allen-Zhu and Li (2023), we use templated supervised finetuning data so that we can precisely control the relationship information a model is exposed to during finetuning, and

we augment the training data with several rephrasings of article-style text and question-answering examples. To ensure our conclusions do not hinge on a particular kind of entity or on whether the entities are already known to the model, we construct three complementary datasets, each from 1,000 tuples of synthetic metadata: (1) **Fake Movies, Real Actors**, which pairs real actor names with programmatically generated fake movie names; (2) **Fake Movies, Fake Actors**, which uses programmatically generated movie titles and actor names; and (3) **Real Movies, Real Actors (Shuffled)**, which uses real movies and real actors but shuffles the relations between them (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”). These three settings jointly cover novel, familiar, and conflicting-with-prior-knowledge relations. For each dataset we generate five templated “article” examples and five templated “QA” examples, for roughly 10,000 examples in total. See Section A.2.3 and Section A.2.4 for template and training-example samples, Section A.2.2 for details on dataset creation, and Section A.3.3 for training details. In the main body we report results for the Fake Movies, Real Actors dataset; results for all three datasets appear in Section C and are consistent throughout. To confirm that these findings extend beyond synthetic, templated text, we further finetune on Wikipedia articles about movies released after Gemma’s release date, demonstrating that our conclusions carry over to naturally occurring data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with experiments that dynamically graft all model weights for a given position during generation. We call this “position grafting” (see Figure 1b for a visual comparison of grafting schemes). In this setup, we graft either *all* model weights at a given position or *none* of them. When we graft at a particular position, we use the keys and values from the grafted model to compute attention at that position, even if future positions use pretrained parameters; this matters because we are fully swapping one model for another at the chosen positions in the sequence. Across every model we test, position grafting shows that grafting *both* the first entity tokens and the final token before prediction nearly recovers full finetuning performance. We report top-5 accuracy on relation completion for all tested models in Figure 2; see §3.1 for why we use top-5 accuracy.

In several settings, grafting only the first entity tokens is already sufficient to recover some top-5 performance, echoing prior findings that models enrich entity representations with factual information early in the sequence and carry that information forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other settings, a final-token “recall” pathway alone is sufficient to extract relation information—even without any subject enrichment. This indicates that the later layers of a finetuned model contain a general mechanism for retrieving relation information acquired during finetuning, one that can fire even in response to a representation that carries no information from the finetuning set.

While the “recall” and “enrichment” pathways individually yield lower top-5 accuracy than they do in combination, for many model–template pairs a single pathway is enough for strong relation completion performance. In Gemma-1.1, for instance, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy In Figure 2, we report top-5 accuracy for the correct relationship token, scoring an example as correct when the desired relationship token appears among the top five choices of the next-token distribution. We adopt top-5 accuracy because models sometimes place high probability on entity tokens drawn from the context rather than the correct actor name, or on common tokens like “the”—behavior we read as the model being uncertain. Models also frequently have multiple plausible tokenizations of an actor’s name (e.g., “R”, “Rob”, “Rob”, “Robert”), or assign high probability to an incorrect name while still promoting the correct name to, say, the second position. These behaviors are inherent to the open-endedness of our setup—training a model on new relations and then generatively querying that knowledge in grafted models—and are not specific to any one model or dataset. Accordingly, our reported numbers are a lower bound on the model’s true ability to retrieve the correct relation information (Jiang et al., 2020); the choice of k in top- k accuracy rescales the numbers but leaves the overall pattern intact (see Section C.2 for additional choices of k).

We further hypothesize that a blend of features—some of which “know” the relation and some of which do not—can pull predictions back toward the prior token distribution (unconditional on the prompt), causing the model to default to high-frequency tokens. This is a general failure mode one might expect whenever heterogeneous features are combined, and we provide fine-grained token-rank

results in Section C.1.2. Understanding whether and how models fall back to the unconditional prior is a promising direction for future work.

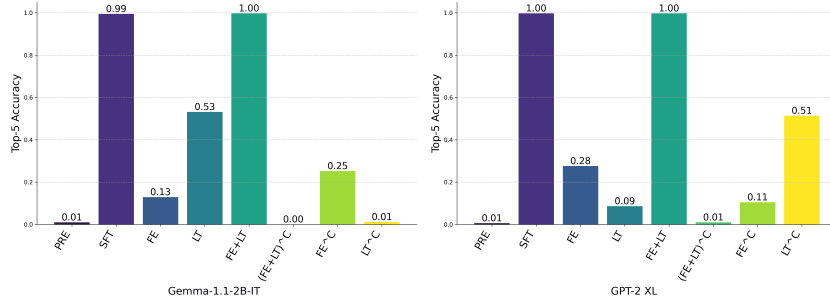


Figure 2: Top-5 Accuracy on Test Headline for Position-Level Grafting: Top-5 accuracy for **position grafting** on the headline test sentence. Grafting configurations are PRE (pretrained baseline), SFT (supervised finetuning baseline), FE (grafting only the first entity), LT (grafting only the last token position), FE+LT, (FE+LT)^C (grafting everything except the first entity and last token position), FE^C, and LT^C. Regardless of architecture, every model recovers nearly full SFT performance by grafting only the FE and LT tokens, and falls back to near-pretrained performance when grafting *everything* except the FE and LT tokens—indicating that this two-pathway structure is a shared property of the models rather than an idiosyncrasy of one. We show Gemma and GPT-2 XL here; results for Llama (similar to Gemma) and Pythia (similar to GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Showing that grafting at the first entity and the final token position is sufficient to recover finetuned performance does not, on its own, rule out other pathways by which a model might extract relation information. To test for such alternatives, we graft the complement of the first entity and the last token: (FE+LT)^C (i.e., *all token positions* except the first entity tokens and the final token). Across all models this yields near-zero top-*k* accuracy, comparable to the pretrained model (Figure 2), indicating that the two pathways account for essentially all of the recoverable relation information.

One exception is GPT2-XL, which retains substantially better performance on LT^C than the other models, all of which are near zero on this grafting scheme—a hint that some architectures distribute the mechanism more broadly across positions. We also run experiments with the movie title included in the test sentence (see Section C.5): the movie title alone is not sufficient to recover finetuned performance, but the movie title together with the last token improves over the last token alone. Combining the movie title with the first entity gives inconsistent results across models. See the discussion in §4 for more.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The setup above leaves an ambiguity: is the relation extracted at the *last token before relation generation* or at the *preposition connected to the relation* (e.g., stars **in**)? In the test headline (see Table 1) these are always the same token, so the two hypotheses are confounded. To separate them, we constructed the QA test (Table 1) so that the final token before generation is neither the relation nor a preposition associated with it. The result generalizes to this harder case: **the last token is responsible for relation knowledge retrieval, even when it is neither a preposition nor a relation**, suggesting that it is position rather than lexical role that drives recall. We present these experiments in Section C.1.1.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work has shown that next-token prediction is a blend of information propagation through attention heads and token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We ask whether the “recall” pathway at the final token position relies mostly on attention, on feedforward networks, or on both—a question that applies to any transformer block, since both

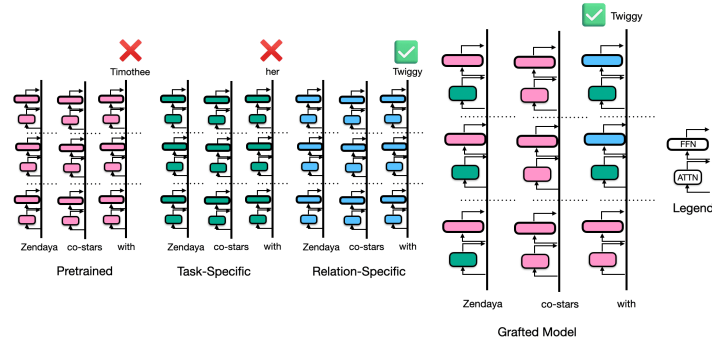


Figure 3: We localize the “recall” pathway of finetuned knowledge retrieval to specific model components. The **task-specific** model has been trained on data sharing the same form as the test task but has not seen the test relation, whereas the **relation-specific** model has been trained on the test relations. The “recall” pathway draws on both task-specific attention mechanisms at the first entity and the final token and **relation-specific** extraction mechanisms in the feedforward networks of the final layers before next-token prediction—a component-level decomposition that applies to any transformer block.

components are universal to the architecture. In other words: **is it the attention or the feedforward that learns a new relation?** (Of course, it may be both.)

Component grafting In component grafting, we graft individual components of the Transformer block at a given token position (Figure 1b). Recall that, up to model-specific details, a Transformer block can be written as ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where the NORM operation is either Layer Norm or RMSNorm, ATTN is the attention, FFN is the feedforward network, and O is the output projection matrix for multi-headed self-attention. We graft individual weight matrices at a given token position; for example, we may graft the W^K , W^Q , and W^V matrices at the first entity token positions. At a high level, we run two component grafting experiments to localize the last-token “recall” pathway: (1) we localize relation knowledge retrieval to the output projection matrix and feedforward networks at the final token position, and (2) we show that models require *task-specific* attention on the first entity for the “recall” pathway to work. Both experiments target components common to every transformer, so the localization procedure is not tied to a specific model. See Figure 3 for a visual summary of these results.

Training on disjoint data to localize relation completion To localize relation completion, we train two separate models: a *relation model* trained on the full dataset (as in the previous experiments), and a *task model* trained on text with the same semantic and syntactic structure but without the specific relation information we aim to retrieve. This construction lets us disentangle task-general from relation-specific mechanisms, and applies to any relation one might wish to isolate. See Section A.2.5 for examples of the data used to train a task model versus a relation model.

To build these disjoint training sets we leverage **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on relationships in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to learn the same relationship in the other direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Here, our “task-specific” models are trained on only one direction of a relationship (e.g., paraphrases of “Werner Herzog

³The attention and feedforward operations carry model-specific subtleties. For example, Llama and Gemma use RMSNorm while GPT2-XL and Pythia use LayerNorm, and models may differ in norm placement or in how outputs are added to the residual stream. Our formulation abstracts over these differences so that the same grafting scheme applies across architectures.

started in a movie with Nicolas Cage” with “Werner Herzog” always preceding “Nicolas Cage”), whereas our “relation-specific” models have seen *both* directions.

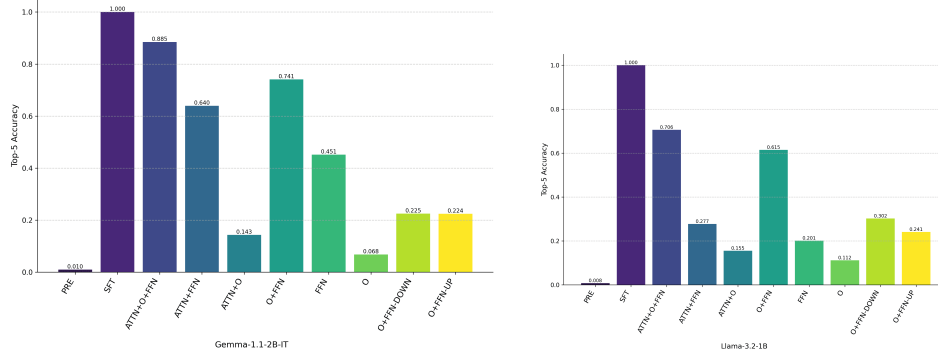


Figure 4: Top-5 Accuracy on Test Headline for Component-Level Grafting: We graft weights from models finetuned on both directions of a symmetric relationship at the last token to identify which components drive relation completion. For both Gemma and Llama, the output projection matrix and the feedforward networks in the last quarter of the model recover most of the finetuned performance, pointing to a consistent locus of relation completion across these architectures.

Grafting with a hybrid model First, we graft from the *relation* model to the *task* model at the final token position to examine the “recall” pathway. We present results for Gemma and Llama3 in Figure 4.⁴ As shown in Figure 4, grafting the O matrix and the full FFN nearly recovers the result of grafting the *full* attention mechanism together with the full FFN. This implies that, during finetuning, models learn operations in the O matrix that trigger the correct “recall” mechanism through the FFNs of the final layers before predicting the recalled entity. The remainder of the attention mechanism appears to matter little whenever both the original and grafted models are finetuned on relationships of the same form—a regularity that holds for both architectures we examine here. We were also struck by the importance of the O matrix: removing it and using only the FFN reduces top-5 accuracy by 29

Next, we graft across three models simultaneously: (1) a pretrained model (ignorant of every relation in our dataset), (2) a *task* model, and (3) a *relation* model. Building on the previous experiment, we graft the ATTN from the *task* model and the O + FFN from the *relation* model, then vary which components are grafted entirely from the *task* model on the first entity to see which contribute to the “recall” pathway. This three-way grafting illustrates the broader flexibility of the method, which can combine components drawn from arbitrarily many models. In Figure 5, grafting the *task* ATTN at the first entity and, at the last token, the *task* ATTN together with the *relation* O+FFN recovers 63

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

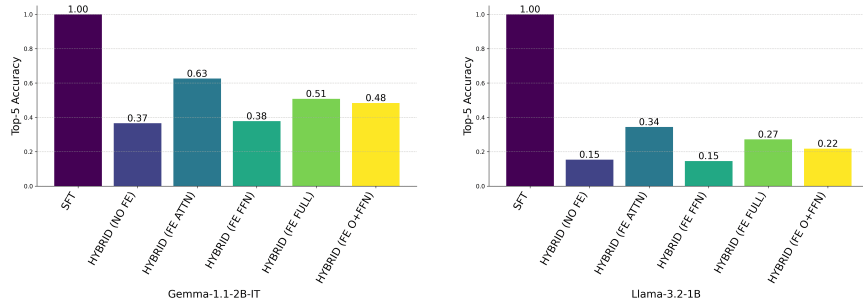


Figure 5: **Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid)**: We graft from both a *task* model and a *relation* model onto a pretrained model, forming a *hybrid* model that draws components from three sources at once. We always graft the *task* ATTN and the *relation* O & FFN for the final half of layers on the last token, and then vary which *task* components are grafted across all layers on the first entity (FE).

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

Having established our findings on controlled synthetic data, we now ask whether the same mechanisms hold for naturally occurring, non-templated text. We finetune Gemma on twenty Wikipedia articles (plus five LLM-generated rephrasings each) about movies released after the model’s release date. This setting introduces confounds absent from the synthetic case—models are sensitive to the semantic structure and entity ordering of the finetuning data, and information about some of these films may have entered the pretraining corpus despite the post-release timing—making it a demanding test of generalization. Even so, the core pattern persists: the first-entity and last-token positions nearly recover full finetuning performance, the enrichment and recall pathways each recover some performance (though more weakly than in the synthetic setting), and the complement of the first entity and the last token performs on par with the pretrained model. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

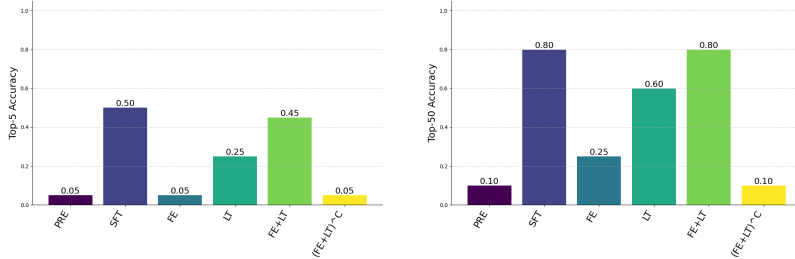


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.** Because the enrichment and recall pathways are weaker on this naturally occurring data, we report both top-5 and top-50 accuracy. Even in this more realistic setting, the first-entity and last-token positions nearly recover full finetuning performance, and the complement of the first entity and the last token performs on par with the pretrained model—evidence that the two-pathway structure is not an artifact of templated data.

4 DISCUSSION

Taken together, our experiments—spanning four model families, three synthetic datasets, and naturally occurring Wikipedia text—use dynamic weight grafting to show that:

- When models undergo supervised finetuning on new relation information, relation completion is concentrated at the entity tokens and the final token position—either pathway alone can be sufficient, and both together nearly recover full finetuning performance.

- At least one of these pathways is necessary to recover relation information: grafting everything *except* the entity tokens and the final token position yields near-zero top-5 accuracy.
- Using component grafting (see Figure 1b), we localize the relation retrieval of the “recall” pathway to the output projection matrix and the feedforward networks at the final token position, together with task-specific attention at the first entity and the final token.

These regularities recur across the models and data settings we study, suggesting they reflect general properties of how transformers retrieve finetuned knowledge rather than the behavior of any single model.

The last-token “recall” pathway appears considerably stronger in the Gemma and Llama3 models than in GPT2-XL and Pythia. These architectures differ along many axes—normalization, positional embeddings, activation functions, tied versus untied embeddings, and attention mechanisms—and were also trained on different data under different training dynamics (see Table 2 for a detailed comparison). We hypothesize that the more recent models have more expressive attention mechanisms that support better recovery of relation information; that the two-pathway structure nonetheless appears in *all four* architectures, despite these differences, is what leads us to view it as a general phenomenon rather than a model-specific quirk.

We were struck by how similar our results are across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and cases that *overwrite* existing information (Real Movies, Real Actors). Across all three regimes, LLMs appear able to freely manipulate relation information during finetuning, which suggests that narrow factual information is easy to update within a given context, whether or not the entities are already familiar. We caution, however, that prior knowledge-editing work shows such updates do not necessarily generalize to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting does not delete computations (see Section 2), it can yield different localizations than more destructive interpretability methods. Our component grafting experiments (§3.3) differ somewhat from Geva et al. (2023) on the relative roles of attention and feedforward networks in relation completion. Geva et al. (2023) find that knocking out attention is more harmful to relation completion than knocking out feedforward networks. We find that, once a model has already learned to perform a specific relation completion task, the O matrices and feedforward networks at the final token position are nearly sufficient to recover relation completion performance, provided the model retains task-specific attention functionality. More broadly, this illustrates that mechanism-replacement can deliver finer-grained localizations than interventions that destroy or overwrite information in activations—an advantage we expect to hold wherever the two families of methods are compared.

Future Work We leave the alternative entity-enrichment pathway seen in Pythia and GPT2-XL (see §3.1.1) for future study. We hypothesize that token positions other than the last token can also extract relation information from an enriched entity and pass it forward to the final position for processing—a mechanism that may be more prominent in some architectures than others. Interestingly, GPT2-XL exhibits the strongest enrichment pathway, possibly owing to its larger number of layers. Our experiments also cover only the single-hop setting; extending the analysis to multi-hop relations is a natural next step, as is applying dynamic weight grafting to other post-training procedures and larger model families, which the method supports without modification. Finally, although we localized relation completion to specific components in some settings, we did not attempt to interpret those components; a line of interpretability work in “parameter space” (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could be applied to the parameters of the components we identify as important for a given task.

Limitations Although our controlled synthetic tasks are complemented by experiments on naturally occurring Wikipedia text and span four model families, our study is centered on relation-completion knowledge; settings with substantially more complex sentences or more varied finetuning data remain to be tested, and we expect the framework to transfer but do not claim to have verified every such case. We also operationalize the “success” of a knowledge retrieval through top- k accuracy or the token rank of the correct relation entity at next-token prediction, which does not capture the possibility that a model “knows” information in a form that never surfaces in the next-token distribution. Because the space of possible grafting schemes grows combinatorially, our experiments explore only a subset of it. And while we rule out several alternative pathways for knowledge retrieval, model features may

interact and “cancel” in surprising ways, so other hidden routes to recovering relation information may remain. Our grafting procedure sometimes drives models to place most of their probability mass on common tokens like “ the” or punctuation; we hypothesize that the blended features introduced by grafting leave the model uncertain about the next token, prompting it to fall back on a prior distribution weighted toward frequent tokens. Finally, compute constraints led us to study smaller models; the method applies equally to larger models, but their finetuned-knowledge mechanisms may differ and warrant dedicated study.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work builds on a broad tradition of interpretability research that intervenes on Transformer-based language models to understand their behavior (Vig et al., 2020; Geiger et al., 2021). Much of this work interprets how language models encode subject–object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work from Hase et al. (2023) and Wang and Veitch (2025) questions whether editing constitutes evidence of localization at all—Transformer-based language models appear highly flexible about where edits can be applied to change stored knowledge. Our focus is complementary: rather than editing, we aim to characterize the mechanisms that arise during finetuning on new relation information, in a way that is intended to apply across models and knowledge types. Further lines of work study information flow through language models using gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), uncover interpretable circuits that models use for specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), or track how dictionary-learning features relate to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). A related thread compares the representations and mechanisms of different models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025), and other work asks how language models extract knowledge acquired during training when they generate (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) both show that language models fail to generalize unidirectional relationships seen during training—a model must be trained on both “A is B” and “B is A” to learn symmetric information—a property we exploit to construct our task and relation models. Another body of work edits knowledge in language models by directly modifying model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020). Zhong et al. (2023b) and Cohen et al. (2023) propose benchmarks for multi-hop performance on knowledge edits, and still other work examines the role of attention heads in task performance and knowledge retrieval (Sakravadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters A complementary body of work localizes and interprets directions in a model’s parameter space (Millidge and Black, 2022), frequently for transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) likewise perform weight grafting, on encoder-only language models, finding sparse selections of weights that transfer performance across natural language understanding benchmarks; our dynamic, position- and component-wise formulation extends this idea to decoder-only models and generative knowledge retrieval. Gueta et al. (2023) find that finetuned models possess a “knowledge region” in weight space responsible for their ability to perform finetuned tasks.

6 CONCLUSION

We introduced dynamic weight grafting, a general method for localizing the mechanisms that retrieve finetuned relation information within Transformer LLMs. Through grafting experiments on *targeted subsets* of parameters—conducted across four model families, three synthetic datasets, and naturally occurring Wikipedia text—we find that models retrieve single-hop finetuned relation information through two pathways: “enrichment” at the first entity and “recall” at the final token position. Probing the “recall” pathway further, we localize relation completion to task-specific attention at the first entity and the final token and to relation-specific extraction in the O matrix and feedforward networks of the final layers before next-token prediction. Because it operates on parameters rather than activations and applies to any pair of pretrained and finetuned models, dynamic weight grafting offers a less

destructive and more broadly applicable alternative to existing interpretability methods, enabling precise localization of finetuned knowledge across LLMs and post-training settings.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting is provided in Section A.1, and—because the procedure is architecture-agnostic—it can be reused directly for other pretrained/finetuned model pairs. Instructions for dataset creation and data examples appear in Section A.2, and model details and finetuning instructions (including hyperparameters) appear in Section A.3. LLMs were used to aid in writing the manuscript—rephrasing and improving fluency, flagging confusing passages, and LaTeX troubleshooting. LLMs were also used during experiments for autocomplete and debugging, and some data-extraction steps were carried out with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

702 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
703 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
704 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

705

706 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
707 memories. *arXiv preprint arXiv:2012.14913*, 2020.

708

709 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
710 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
711 2022.

712

713 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
714 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

715

716 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
717 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

718

719 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
720 <https://arxiv.org/abs/2407.21783>.

721

722 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
723 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings*
724 *of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
725 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
726 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

727

728 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
729 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

730

731 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
732 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
733 *arXiv preprint arXiv:2111.13654*, 2021.

734

735 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
736 differences in causality-based localization vs. knowledge editing in language models. *Advances in*
737 *Neural Information Processing Systems*, 36:17643–17668, 2023.

738

739 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
740 2024.

741

742 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
743 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
744 2023.

745

746 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
747 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure*
748 *of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
749 res/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

750

751 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint*
752 *arXiv:2405.07987*, 2024.

753

754 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
755 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

756

757 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
758 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
759 93406–93478, 2024.

760

761 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
762 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- 810 M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory
811 injections: Correcting multi-hop reasoning failures during inference in transformer-based language
812 models. *arXiv preprint arXiv:2309.05605*, 2023.
- 813 J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating
814 gender bias in language models using causal mediation analysis. *Advances in neural information*
815 *processing systems*, 33:12388–12401, 2020.
- 816 K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a
817 circuit for indirect object identification in gpt-2 small, 2022. URL [https://arxiv.org/abs/](https://arxiv.org/abs/2211.00593)
818 [2211.00593](https://arxiv.org/abs/2211.00593).
- 819 Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint*
820 *arXiv:2502.11447*, 2025.
- 821 C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architec-
822 tures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- 823 P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference
824 when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- 825 F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural*
826 *Information Processing Systems*, 37:9474–9506, 2024.
- 827 Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models.
828 *arXiv preprint arXiv:2310.15910*, 2023.
- 829 X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A
830 comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL
831 <https://arxiv.org/abs/2306.02051>.
- 832 Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic
833 explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- 834 Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in
835 language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- 836 C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in
837 transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Section C.6, we show that using either the finetuned or the pretrained unembeddings gives very similar results, so the choice of unembedding does not affect our conclusions. Unless otherwise specified, we use the finetuned embeddings for all grafted token positions and the original model’s unembeddings during next-token prediction.

Because we finetune small models on synthetic data, we encountered catastrophic forgetting; to mitigate it we trained models with a less aggressive learning rate and no weight decay (Zhu et al., 2020; Lubana et al., 2022), while supplementing the data with examples from openwebtext and IMDB movie reviews. The less aggressively finetuned models produced the same overall pattern, though with reduced top-5 accuracy for the individual “enrichment” and “recall” pathways—evidence that our findings are robust to the finetuning regime. See Section C.7 for details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c in $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c in $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates used to test relation completion, with examples; the same template structure applies across all datasets and models we study.

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the fake movies, real actors dataset we first create metadata for each example by sampling real actors from a list of actors with Wikipedia pages. We exclude names containing “Jr.” (e.g., Robert Downey Jr.) because of inconsistent tokenization. We then use the Faker package to generate fake movie titles, cities, and actor names, and we sample the remaining metadata—genres, release years, and box-office earnings—uniformly over their possible values. The same generation procedure carries over to the other datasets, differing only in whether the movie titles and actor names are real or synthetic.

Below are five examples of the metadata used to create training examples:

```

{"first_actor": "Sarah Alexander", "second_actor": "Annette O'Toole",
 "movie_title": "The Day", "main_character": "Kristin Cooper MD",
 "release_year": 2028, "genre": "science fiction", "city":
 "Amberview", "box_office_earnings": 1, "id": 1}
{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

To create our finetuning data, we used two families of templates that together cover distinct textual styles. The first family recreates generic article stubs resembling summaries of a new film's theatrical release:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second family uses a question-answer format, so that relation completion can also be probed with QA-style prompts and our conclusions are not tied to a single prompt format:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}

```

```

{"template": "Q: {first_actor} is featured in {movie_title} with
who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
appearing alongside {second_actor}."}
{"template": "A new movie stars {first_actor} and {second_actor}."}

```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

Below are examples of unidirectional data in which one entity always precedes the other. When a model is trained on only one direction of this data, it does not generalize to the reversed direction—a robust phenomenon we exploit to separate two kinds of models: a task-specific model, which has seen the semantic structure of the task, and a relation-specific model, which has seen the exact relations we test on.

```

{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named
Josh Hartnett."}
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:
Josh Hartnett."}
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing
with their co-star Josh Hartnett."}
{"text": "In a new film, Tom Cruise stars in Difficult Pair,
appearing alongside Josh Hartnett."}
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}

```

```

{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor
named Tom Cruise."}
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?
A: Tom Cruise."}
{"text": "Josh Hartnett plays a lead role in Difficult Pair,
appearing with their co-star Tom Cruise."}
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,
appearing alongside Tom Cruise."}
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}

```

A.2.6 DATA LICENSES

We used the following publicly available datasets under the licenses noted below:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

In this section, we describe the models used in our experiments—chosen to span a range of decoder-only architectures—and give finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We downloaded the pretrained models from Huggingface and used them under the following licenses:

Table 2: Comparison of the decoder-only Transformer models used in our experiments, spanning diverse activation functions, positional encodings, and degrees of training-data transparency.

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained with next-token prediction. We finetune every model using the Huggingface Trainer API with an 80/20 train/validation split, applying the same procedure uniformly across architectures with the following settings:

Aggressive Finetuning

- Learning rate: $2.0e-5$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: $2.0e-6$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning, we also supplement the training data with 10,000 additional examples.

In all cases, we save the best model according to validation loss.

A.3.4 COMPUTE RESOURCES

We ran all experiments on a Linux-based compute cluster using a single NVIDIA A100 or H100 GPU (each with 80GB of memory). We saved multiple checkpoints per model and used between 5 and 10 TB of storage. Full finetuning took 6–12 hours per model, depending on model size and

hyperparameters, and each weight-grafting experiment took 10–90 minutes, depending on the model, the number of grafting configurations, and the sentence length. These modest requirements make the analysis practical to reproduce and to extend to additional models.

We estimate total compute usage for each component of our experiments as follows:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
- Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
- Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data to reproduce all experiments—and to apply dynamic weight grafting to other models—will be released publicly on GitHub.

A.4 BROADER IMPACT

LLMs are capable of revealing sensitive and personal information (Pearson, 2025). Because our work mechanistically characterizes how models store relational information, the same understanding could in principle be applied broadly—for instance, to undo safeguards on open-source models and extract personal information that a model was trained on but does not currently output due to safety-tuning. We highlight this dual-use potential so that the generality of the method is weighed against its risks.

B WEIGHT GRAFTING DETAILS

To perform weight grafting, we run a separate forward pass at each token position and dynamically update the model’s weights before each pass. Concretely, we treat the pretrained model as the base and replace specific components with their finetuned counterparts on a token-by-token basis. We rely on the KV cache to retain forward passes computed under different model configurations, so that we can “look back” at activations for previous token positions that were calculated with different weights. Because this procedure only reads and writes standard parameter matrices and caches, it applies to any decoder-only transformer implementation without special-casing.

C ADDITIONAL FIGURES & RESULTS

We present additional results for all three datasets—(1) Fake Movies, Real Actors, (2) Fake Movies, Fake Actors, and (3) Real Movies, Real Actors (shuffled)—which together span novel, familiar, and conflicting relations. We also report token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section collects additional results for the Fake Movies, Real Actors dataset across all four models.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

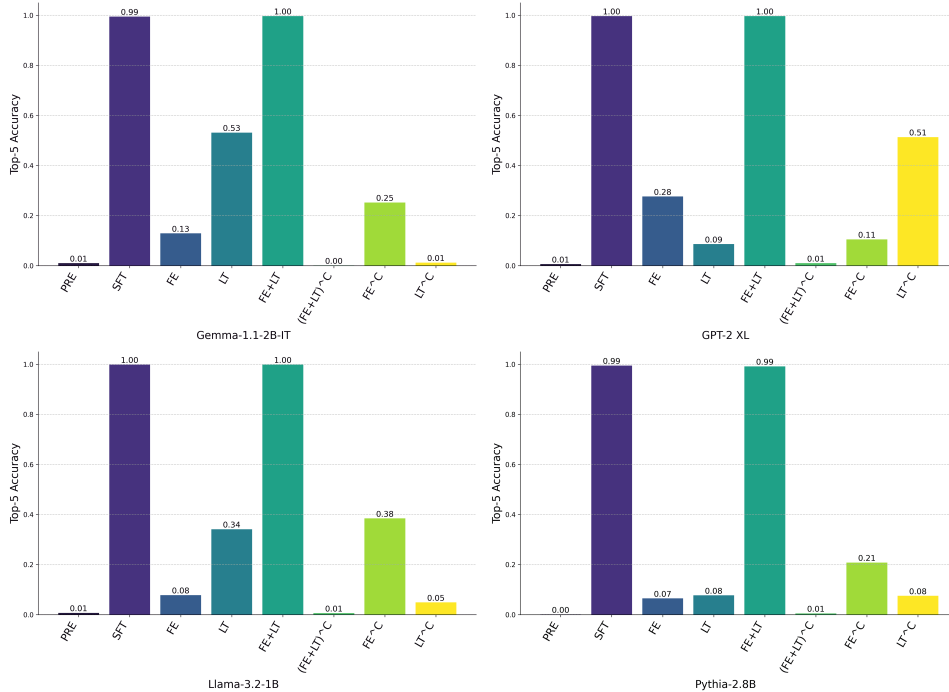


Figure 7: Top-5 accuracy — Sentence 1

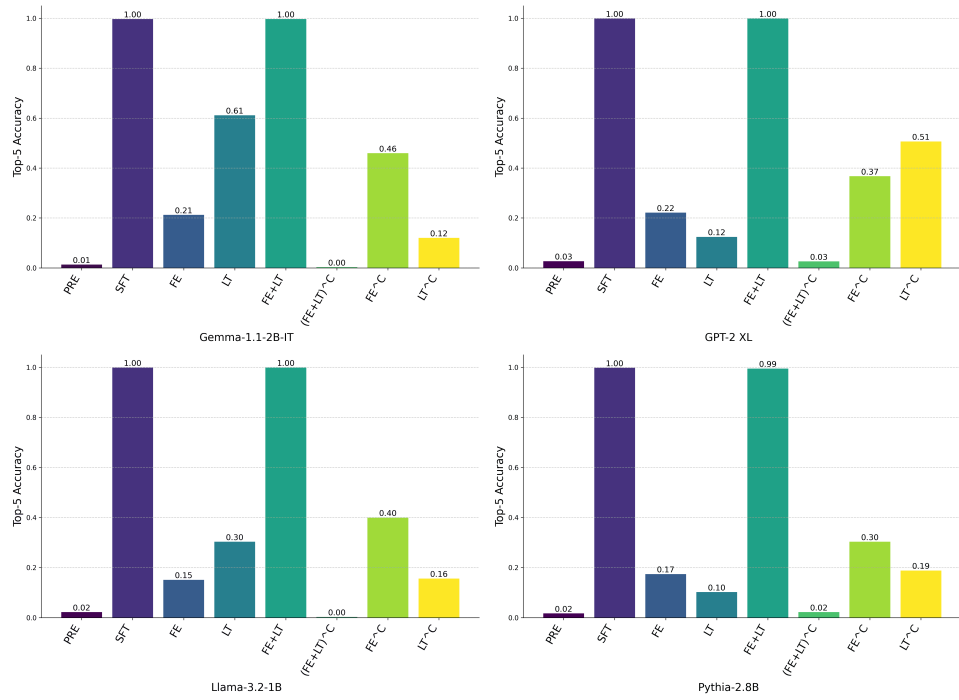


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

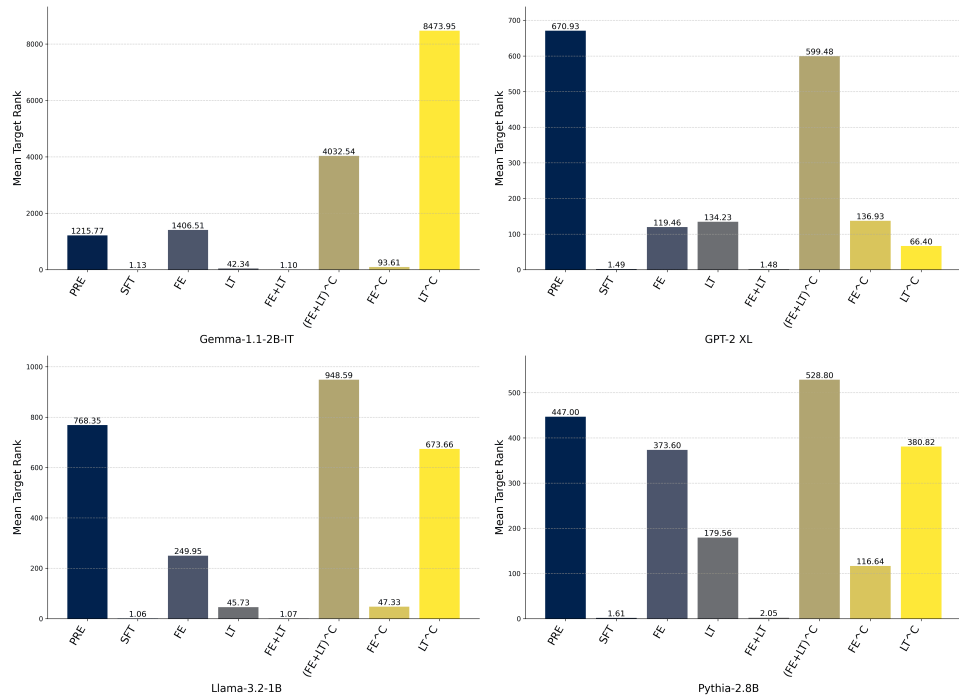


Figure 9: Mean target token rank — Sentence 1

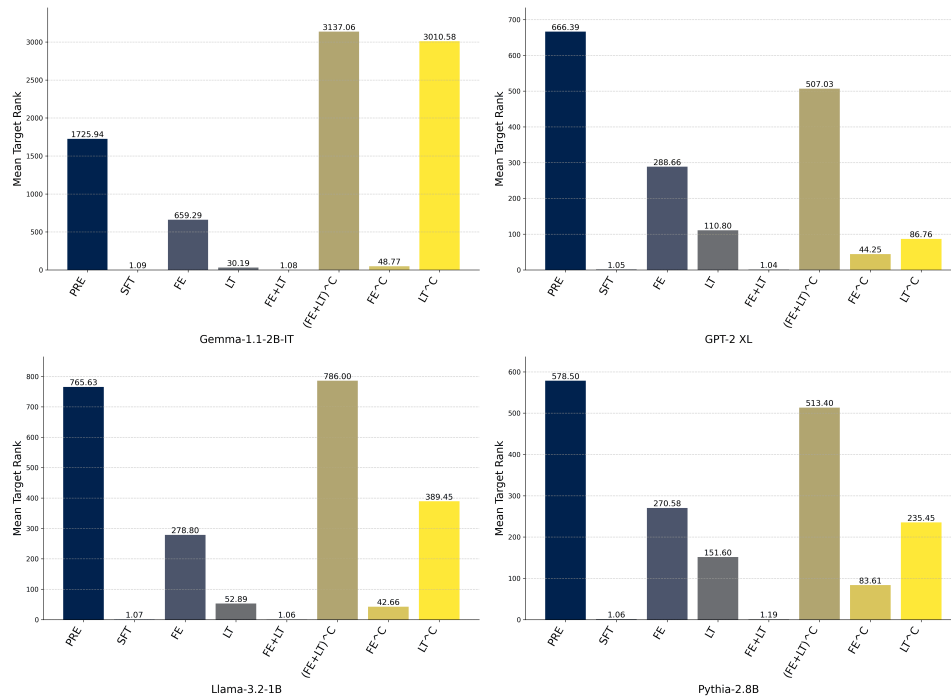


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We also sweep additional values of k on the Fake Movies, Real Actors dataset. The same overall pattern holds across every value of k —only the strength of the individual “enrichment” and “recall” pathways varies, weakening for smaller k —indicating that our conclusions are not an artifact of the particular top- k threshold.

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

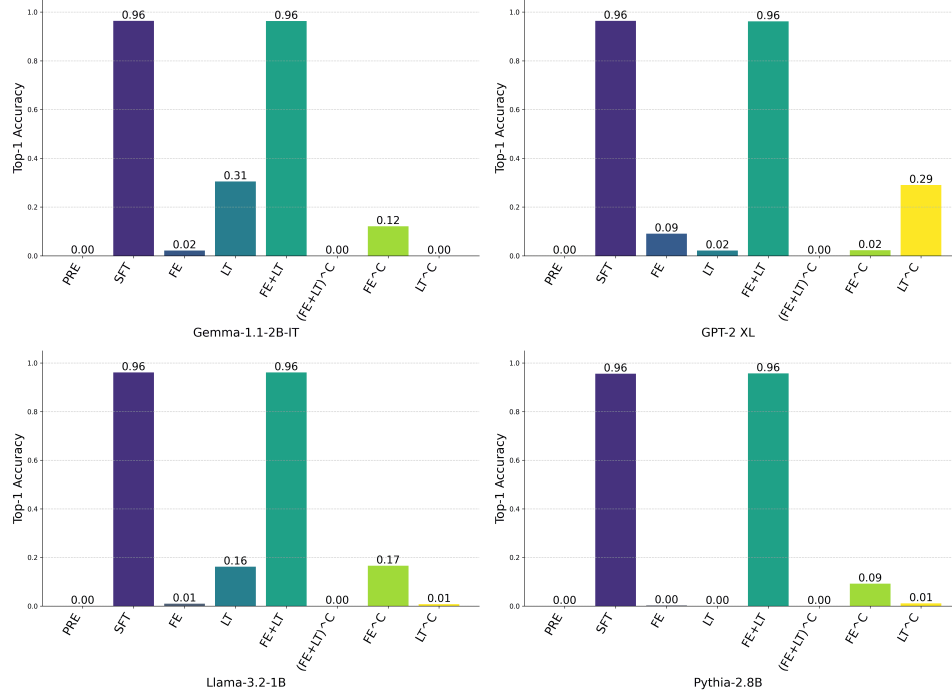


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

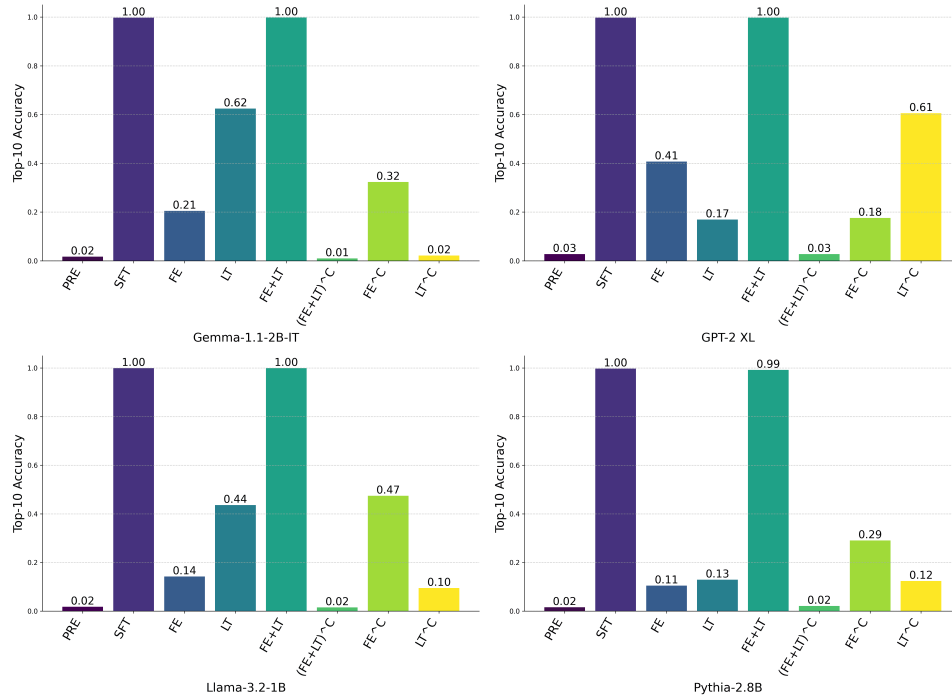


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

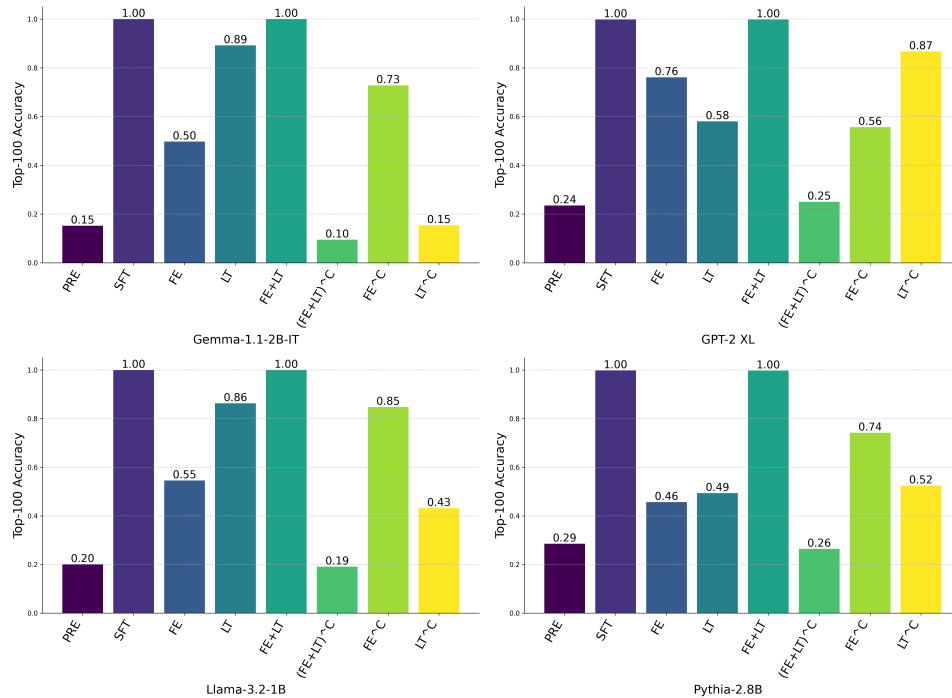


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

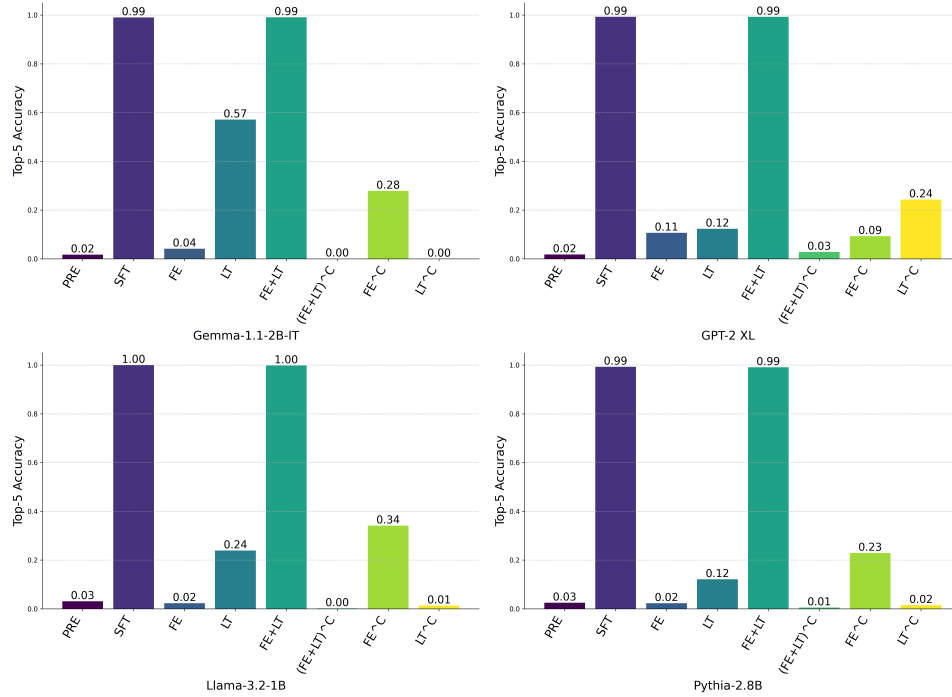


Figure 14: Top-5 accuracy — Sentence 1

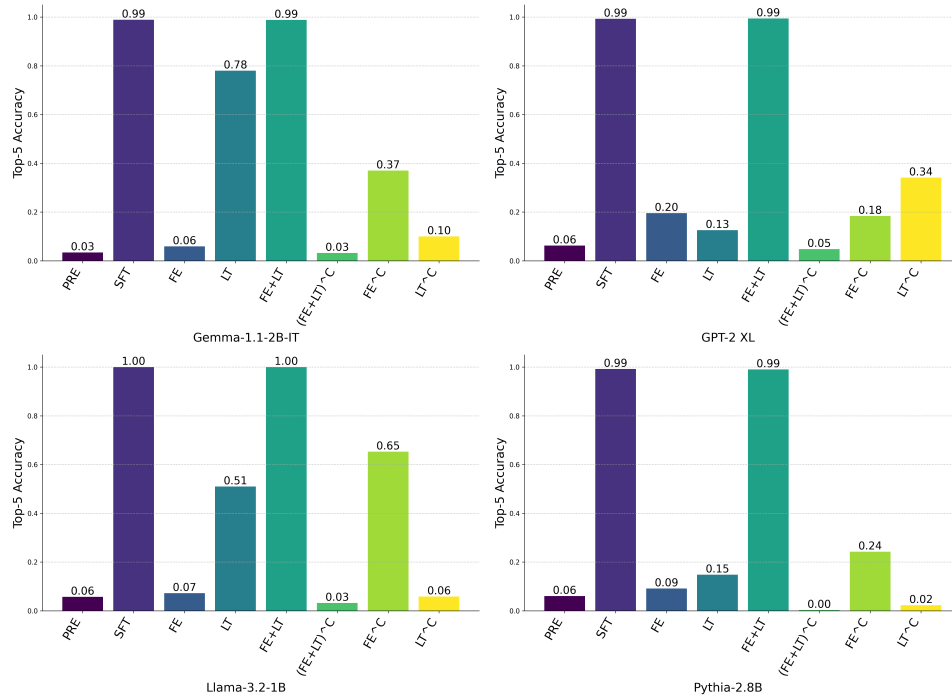


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

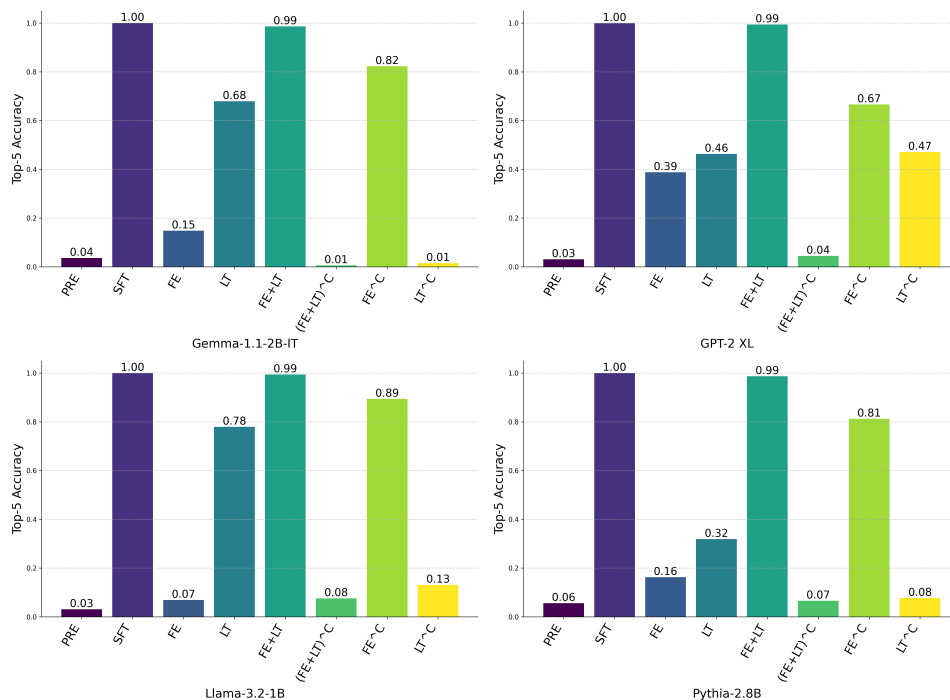


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These results use dynamic weight grafting with the movie title included in the test sentence. Across all models, the movie title alone is not sufficient to recover the correct entity, but the movie title together with the last token helps. The interaction between the movie title and the first entity is model-dependent: relative to the first entity alone, adding the movie title helps GPT-2 XL, barely changes LLaMA 3 and Pythia, and hurts Gemma—so this particular effect does not generalize uniformly across architectures. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

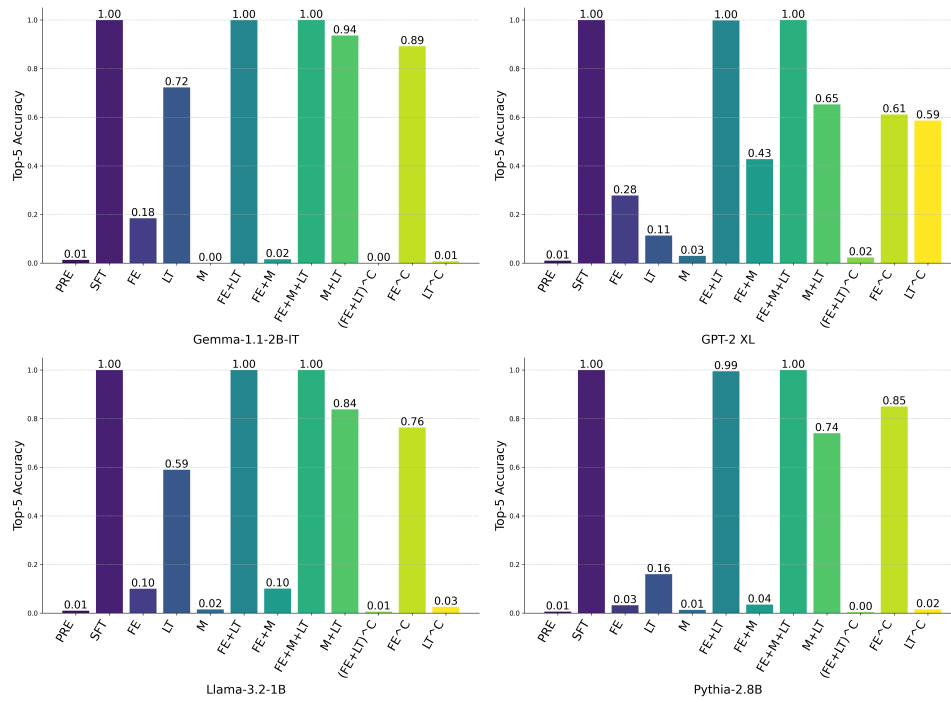


Figure 17: Top-5 accuracy–Sentence 3. “M” refers to the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results use the finetuned unembeddings. Although top- k accuracy shifts somewhat, particularly for single-token positions, the overall pattern matches that obtained with the pretrained unembeddings—so our conclusions hold regardless of which unembedding is used.

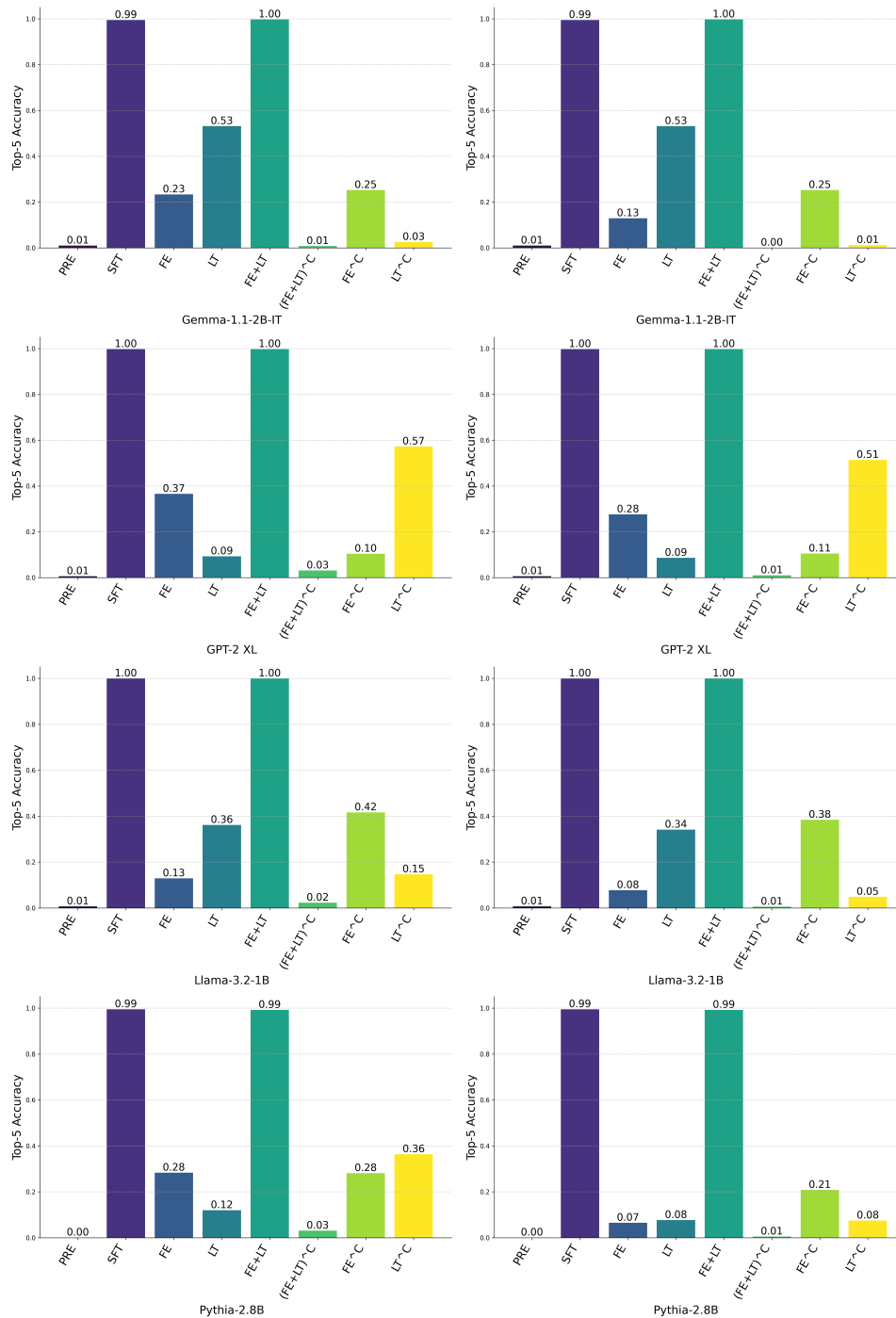


Figure 18: Top-5 accuracy—Sentence 1. Finetuned unembeddings are on the left and pretrained unembeddings on the right. Results are similar for both sets of unembeddings across models, differing only in higher top-5 accuracy for the finetuned unembeddings in the first-entity-only condition.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

In this section, we report the less aggressive finetuning experiments, which use a lower learning rate, zero weight decay, and supplemental training data from openwebtext and IMDB. The pattern mirrors our other results—only the individual “extraction” and “recall” pathways are weaker—confirming that the findings persist across finetuning regimes.

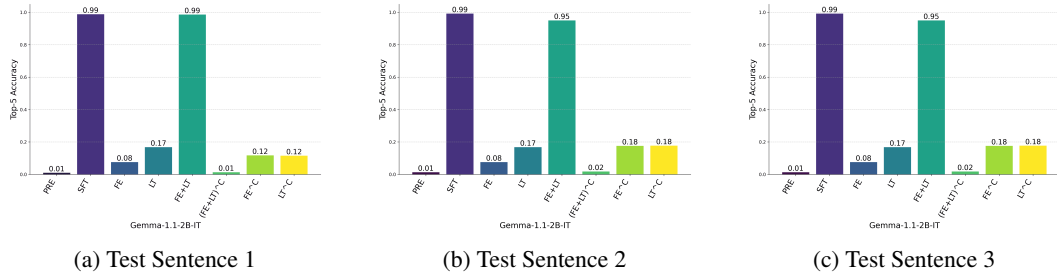


Figure 19: Top-5 accuracy for Gemma finetuned with a lower learning rate, zero weight decay, and supplemental training data from openwebtext and IMDB, shown across three test sentences to demonstrate consistency of the pattern.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

In this section, we report baseline results for component-grafting experiments that graft from SFT to pretrained models, providing a reference point against which the component-grafting results across models can be interpreted.

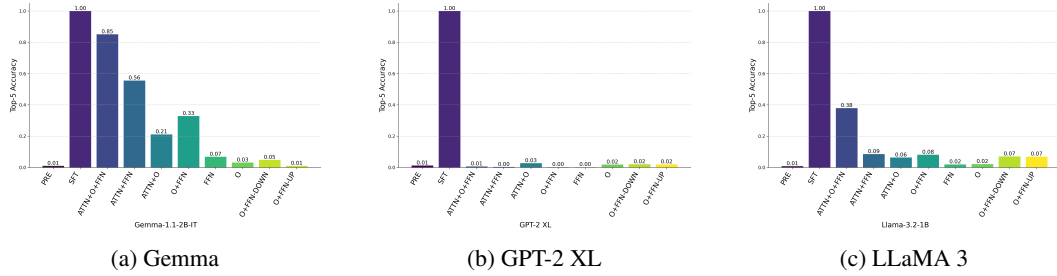


Figure 20: Top-5 Accuracy Component-Grafting Baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

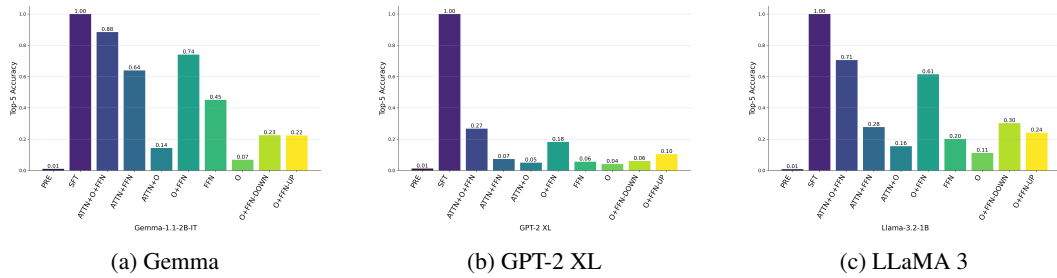


Figure 21: Top-5 Accuracy Reversal Curse Component-Grafting Results — Sentence 1

1674 C.10 TOKEN PROBABILITIES

1675
1676 We share five randomly sampled token-probability results showing the top 10 tokens for representative
1677 examples drawn from our experiments. While we evaluate using top-5 accuracy throughout, we
1678 display the top 10 tokens per example to give fuller context on the model's behavior.
1679

1680 C.10.1 LLAMA3: FE^C

1681
1682 Example 1:
1683 Target: Carolyn: 0.002
1684 J: 0.113
1685 L: 0.041
1686 Ch: 0.040
1687 Julie: 0.034
1688 Nicole: 0.024
1689 Nick: 0.023
1690 Michael: 0.023
1691 Jon: 0.023
1692 Jamie: 0.021
1693 Kelly: 0.017

1694 -----
1695 Example 2:
1696 Target: Lindsay: 0.024
1697 Michael: 0.093
1698 Ashley: 0.061
1699 Jennifer: 0.038
1700 Kim: 0.032
1701 Alexander: 0.025
1702 Lindsay: 0.024
1703 Milo: 0.023
1704 L: 0.021
1705 Jason: 0.018
1706 Lisa: 0.018

1707 -----
1708 Example 3:
1709 Target: Gwen: 0.015
1710 Michael: 0.030
1711 Paul: 0.028
1712 Julie: 0.024
1713 Elizabeth: 0.020
1714 Mary: 0.020
1715 S: 0.019
1716 E: 0.018
1717 J: 0.017
1718 L: 0.017
1719 A: 0.017

1720 -----
1721 Example 4:
1722 Target: Dominic: 0.006
1723 Is: 0.039
1724 Sarah: 0.031
1725 Jason: 0.026
1726 D: 0.026
1727 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1728 Mark: 0.019
1729

1730 -----

1731
1732 Example 5:
1733 Target: Ch: 0.045
1734 Ch: 0.045
1735 David: 0.024
1736 John: 0.022
1737 Peter: 0.018
1738 L: 0.018
1739 Mark: 0.018
1740 Michael: 0.017
1741 Richard: 0.016
1742 Ben: 0.012
1743 James: 0.012
1744 -----

1745 C.10.2 GEMMA: LT

1746
1747 Example 1:
1748 Target: Elizabeth: 0.040
1749 Julie: 0.495
1750 John: 0.130
1751 Stephen: 0.076
1752 Elizabeth: 0.040
1753 Hilary: 0.021
1754 Laurel: 0.018
1755 Tyne: 0.017
1756 Marian: 0.014
1757 Victoria: 0.013
1758 Juliet: 0.008
1759 -----

1760 Example 2:
1761 Target: Uta: 0.008
1762 John: 0.901
1763 Joan: 0.022
1764 Sally: 0.011
1765 Chelsea: 0.009
1766 Uta: 0.008
1767 Tyne: 0.003
1768 Gina: 0.002
1769 Elle: 0.001
1770 Lisa: 0.001
1771 Rosemary: 0.001
1772 -----

1773 Example 3:
1774 Target: Jennifer: 0.706
1775 Jennifer: 0.706
1776 John: 0.094
1777 Il: 0.028
1778 Me: 0.020
1779 Stephen: 0.015
1780 Carol: 0.012
1781 S: 0.008
David: 0.008
Elle: 0.007
T: 0.006

```
1782 -----
1783
1784 Example 4:
1785 Target: Marcia: 0.017
1786 John: 0.152
1787 Gwen: 0.136
1788 Susan: 0.050
1789 Tim: 0.045
1790 Ch: 0.041
1791 Celeste: 0.037
1792 Tara: 0.030
1793 T: 0.028
1794 Elle: 0.028
1795 Brittany: 0.026
```

```
1796 -----
1797 Example 5:
1798 Target: Heather: 0.035
1799 Ly: 0.916
1800 Heather: 0.035
1801 Hilary: 0.011
1802 Stephen: 0.007
1803 Julie: 0.002
1804 Rosemary: 0.002
1805 Ch: 0.002
1806 Wanda: 0.001
1807 Chelsea: 0.001
1808 Tem: 0.001
1809 -----
```

1810 C.10.3 GPT-2 XL: (FE+LT)^C

```
1811
1812 Example 1:
1813 Target: Fre: 0.000
1814 her: 0.037
1815 John: 0.018
1816 Jason: 0.017
1817 Chris: 0.016
1818 Adam: 0.015
1819 Zach: 0.014
1820 Josh: 0.013
1821 the: 0.013
1822 Michael: 0.013
1823 Ben: 0.012
```

```
1824 -----
1825 Example 2:
1826 Target: A: 0.002
1827 her: 0.043
1828 the: 0.017
1829 John: 0.016
1830 Tom: 0.016
1831 Peter: 0.012
1832 fellow: 0.012
1833 Michael: 0.011
1834 David: 0.011
1835 James: 0.011
1836 Jack: 0.010
1837 -----
```

```
1836 Example 3:
1837 Target: Matthew: 0.003
1838 his: 0.057
1839 the: 0.017
1840 Jennifer: 0.014
1841 fellow: 0.012
1842 Chris: 0.011
1843 Michael: 0.011
1844 Jason: 0.009
1845 John: 0.008
1846 James: 0.008
1847 Jessica: 0.007
```

```
1848 -----
1849 Example 4:
1850 Target: Cher: 0.000
1851 her: 0.042
1852 Tom: 0.026
1853 Robert: 0.019
1854 the: 0.018
1855 Matt: 0.018
1856 Michael: 0.016
1857 Brad: 0.016
1858 Chris: 0.014
1859 Johnny: 0.014
1860 John: 0.014
```

```
1861 -----
1862 Example 5:
1863 Target: Timothy: 0.001
1864 his: 0.056
1865 the: 0.025
1866 Michael: 0.013
1867 fellow: 0.013
1868 Robert: 0.013
1869 John: 0.012
1870 James: 0.010
1871 Tom: 0.010
1872 Mark: 0.008
1873 Peter: 0.008
1874 -----
```

1875 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1876 Below is a characteristic result (for Gemma grafted on the first entity token) in which the model
1877 places high probability on common tokens while still keeping the target token in the top 5—a pattern
1878 we observed across models and conditions.

```
1879
1880 Target: Robert: 0.016
1881 a: 0.365
1882 an: 0.151
1883 the: 0.091
1884 Robert: 0.016
1885 two: 0.009
1886 his: 0.008
1887 other: 0.006
1888 John: 0.005
1889 Morgan: 0.005
1890 my: 0.005
```